Cloud Computing

Automatisation

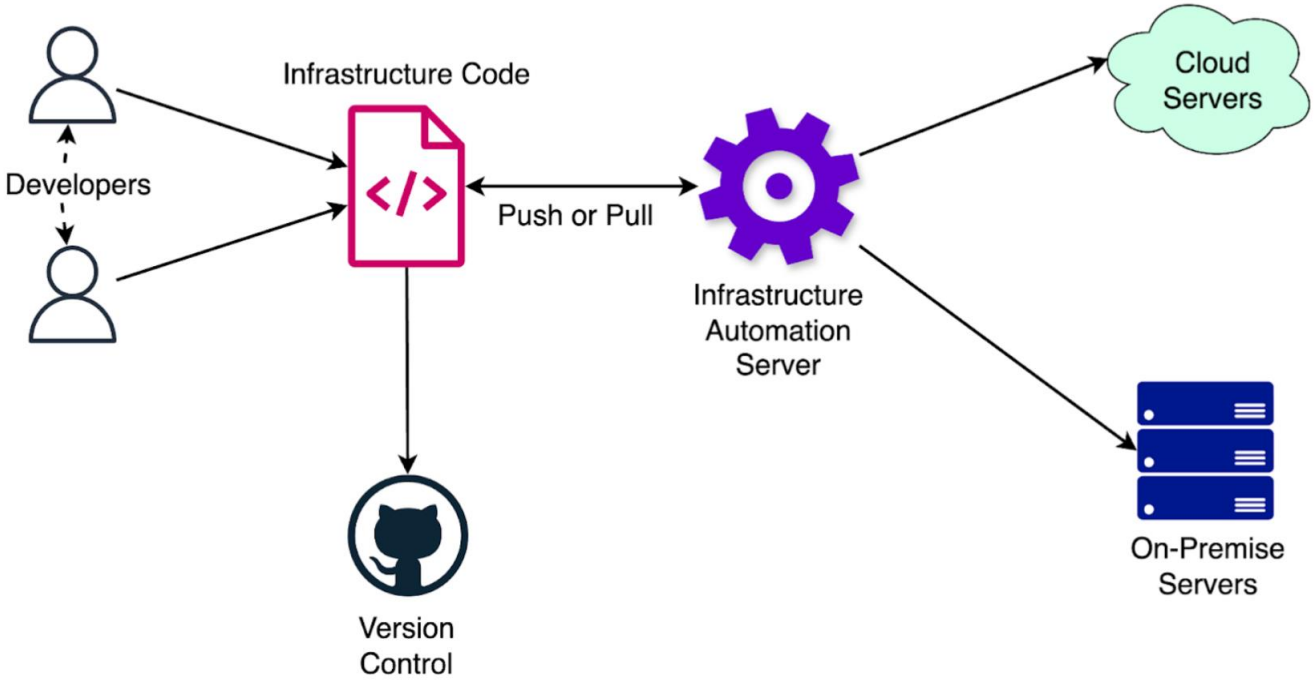
Plan / Automatisation

> Infra-as-Code

> Outillage

> Collaborer sur azure

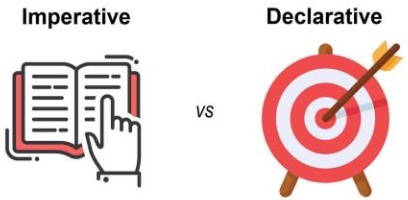
Infrastructure as Code (IaC)



Infrastructure as Code (IaC) -- bénéfices

- Cohérence & répétabilité
- Auditabilité
- Historisation des opérations
- Retour arrière facilité
- Rapidité (minutes vs. jours)
- Auto-documentation
- Collaboration
- Réutilisabilité
- Test & validation
- Optimisation des coûts
- Disaster recovery

Infrastructure as Code (IaC) -- impératif vs. Déclaratif (1/2)

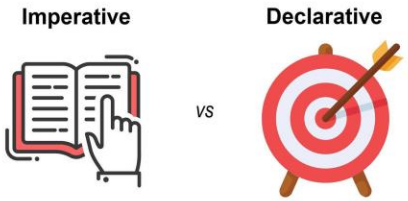


Impératif

- On décrit les opérations à exécuter

```bash
#!/bin/bash
# Imperative: Explicit steps to create infrastructure

# Step 1: Check if resource group exists
if [ $(az group exists -n my-rg) = false ]; then
  echo "Creating resource group..."
  az group create -n my-rg -l francecentral
fi

# Step 2: Check if storage account exists
STORAGE_EXISTS=$(az storage account check-name -n mystorageacct --query nameAvailable)
if [ "$STORAGE_EXISTS" = "true" ]; then
  echo "Creating storage account..."
  az storage account create -n mystorageacct -g my-rg -l francecentral --sku Standard_LRS
else
  echo "Storage account already exists, updating..."
  az storage account update -n mystorageacct -g my-rg --sku Standard_GRS
fi

# Step 3: Create container
az storage container create -n mycontainer --account-name mystorageacct
```

Infrastructure as Code (IaC) -- impératif vs. Déclaratif (2/2)



Déclaratif

- On décrit l'état auquel on veut aboutir

```
# Declarative: Describe the desired state

resource "azurerm_resource_group" "main" {
  name     = "my-rg"
  location = "francecentral"
}

resource "azurerm_storage_account" "main" {
  name                     = "mystorageacct"
  resource_group_name      = azurerm_resource_group.main.name
  location                 = azurerm_resource_group.main.location
  account_tier             = "Standard"
  account_replication_type = "GRS"
}

resource "azurerm_storage_container" "main" {
  name                  = "mycontainer"
  storage_account_name  = azurerm_storage_account.main.name
}
```

Plan / Automatisation

> Infra-as-Code

> Outillage

- **az CLI**
- ansible
- terraform

> Collaborer sur azure

Automatisation / az CLI

- Outil en 'ligne de commande' officiel proposé par Microsoft pour Azure
- Linux, Windows, macOS
- Installation simple
  ubuntu:
  curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
- Initialisation:
  az login –use-device-code
- Premiers pas:
  https://learn.microsoft.com/en-us/cli/azure/get-started-with-azure-cli?view=azure-cli-latest
- Référentiel des commandes:
  https://learn.microsoft.com/en-us/cli/azure/reference-index?view=azure-cli-latest

https://learn.microsoft.com/en-us/cli/azure/install-azure-cli?view=azure-cli-latest

Plan / Automatisation

> Infra-as-Code

> Outillage

- az CLI
- **ansible**
- terraform

Automatisation / ansible

- Travail sur un ensemble de cibles (***inventaire***)
- **Idempotent**
- Garantit le **séquencement** des actions
- **Parallélise** chaque action sur plusieurs cibles
- **Structuration** du code (plays, roles, tasks , ...)
- **Templating** (jinja2)
- Réutilisation et partage de fonctionnalités (**collections**)
- Utilisation de **YAML**

Ansible

```yaml
- name: Construire un nichoir à oiseaux pour John et Paul
  hosts:
    - John
    - Paul

  tasks:
    - name: Couper du bois à la bonne taille
    - name: Assembler les pièces avec des clous
    - name: Peindre le nichoir
```

L'ordre des tâches est important

Ansible

```yaml
- name: Construire un nichoir à oiseaux pour John et Paul
  hosts:
    - John
    - Paul

  tasks:
    - name: Couper du bois à la bonne taille


    - name: Assembler les pièces avec des clous


    - name: Peindre le nichoir
```

Ansible

```yaml
- name: Construire un nichoir à oiseaux pour John et Paul
  hosts:
    - John
    - Paul

  tasks:
    - name: Couper du bois à la bonne taille
      scie:

    - name: Assembler les pièces avec des clous
      marteau:

    - name: Peindre le nichoir
      pinceau:
```

Modules:
outils près à l'emploi qui peuvent faire le travail pour vous

Ansible

```yaml
- name: Construire un nichoir à oiseaux pour John et Paul
  hosts:
    - John
    - Paul

  tasks:
    - name: Couper du bois à la bonne taille
      scie:
        longueur: 30cm
        largeur: 20cm
```

Paramètres:
instructions spécifiques pour que le module sache quoi faire.

Attention à l'indentation !

Ansible

```yaml
- name: Construire un nichoir à oiseaux pour John et Paul
  hosts:
    - John
    - Paul

  tasks:
    - name: Couper du bois à la bonne taille
      scie:
        longueur: 30cm
        largeur: 20cm

    - name: Assembler les pièces avec des clous
      marteau:
        nombre_de_clous: 10
        taille_des_clous: 4cm

    - name: Peindre le nichoir
      pinceau:
        couleur: vert
        couches: 2
```

Tasks:
Pour décrire ce qu'on veut réaliser

Modules:
Ce qui fait le travail pour vous

Paramètres:
instructions spécifiques pour que le module sache quoi faire

Ansible

```yaml
- name: Construire un nichoir à oiseaux pour John et Paul
  hosts:
    - John
    - Paul

  tasks:
    - name: Couper du bois à la bonne taille
      scie:
        longueur: 30cm
        largeur: 20cm

    - name: Assembler les pièces avec des clous
      marteau:
        nombre_de_clous: 10
        taille_des_clous: 4cm

    - name: Peindre le nichoir
      pinceau:
        couleur: vert
        couches: 2
```

ces 'tirets':
Pour décrire des éléments d'une liste

L'ordre importe et est respecté

Ansible

```yaml
- name: Construire un nichoir à oiseaux pour John et Paul
  hosts:
    - John
    - Paul

  tasks:
    - name: Couper du bois à la bonne taille
      scie:
        longueur: 30cm
        largeur: 20cm

    - name: Assembler les pièces avec des clous
      marteau:
        nombre_de_clous: 10
        taille_des_clous: 4cm

    - name: Peindre le nichoir
      pinceau:
        couleur: vert
        couches: 2
```

D'abord on coupe

Ensuite on assemble

Enfin on peint

Ansible

```yaml
- name: Construire un nichoir à oiseaux pour John et Paul
  hosts:
    - John
    - Paul

  tasks:
    - name: Couper du bois à la bonne taille
      scie:
        longueur: 30cm
        largeur: 20cm

    - name: Assembler les pièces avec des clous
      marteau:
        nombre_de_clous: 10
        taille_des_clous: 4cm

    - name: Peindre le nichoir
      pinceau:
        couleur: vert
        couches: 2
```

Play

Liste de tâches qu'on veut exécuter sur un ensemble de cibles (hosts)

Ansible

```yaml
- name: Construire un nichoir à oiseaux pour John et Paul
  hosts:
    - John
    - Paul

  tasks:
    - name: Couper du bois à la bonne taille
      scie:
        longueur: 30cm
        largeur: 20cm

    - name: Assembler les pièces avec des clous
      marteau:
        nombre_de_clous: 10
        taille_des_clous: 4cm

    - name: Peindre le nichoir
      pinceau:
        couleur: vert
        couches: 2
```

Play

Liste de tâches qu'on veut exécuter sur un ensemble de cibles (hosts)

```yaml
- name: Fabriquer des tasses en argile
  hosts:
    - george
    - ringo

  tasks:
    - name: Façonner l'argile en forme de tasse
      tour:
        terre: argile_rouge
        forme: tasse

    - name: Laisser sécher les tasses
      sechoir:
        durée: 24h
        humidité: basse

    - name: Cuire les tasses dans le four
      four:
        temperature: 1000
        durée: 6h
```

Play

Liste de tâches qu'on veut exécuter sur un ensemble de cibles (hosts)

Ansible

artisanat.yaml

```yaml
- name: Construire un nichoir à oiseaux pour John et Paul
  hosts:
    - John
    - Paul

  tasks:
    - name: Couper du bois à la bonne taille
      scie:
        longueur: 30cm
        largeur: 20cm

    - name: Assembler les pièces avec des clous
      marteau:
        nombre_de_clous: 10
        taille_des_clous: 4cm

    - name: Peindre le nichoir
      pinceau:
        couleur: vert
        couches: 2

- name: Fabriquer des tasses en argile
  hosts:
    - george
    - ringo

  tasks:
    - name: Façonner l'argile en forme de tasse
      tour:
        terre: argile_rouge
        forme: tasse

    - name: Laisser sécher les tasses
      sechoir:
        durée: 24h
        humidité: basse

    - name: Cuire les tasses dans le four
      four:
        temperature: 1000
        durée: 6h
```

Playbook

Liste de "plays" exécutés (dans l'ordre) par Ansible

(rassemblés dans un fichier YAML)

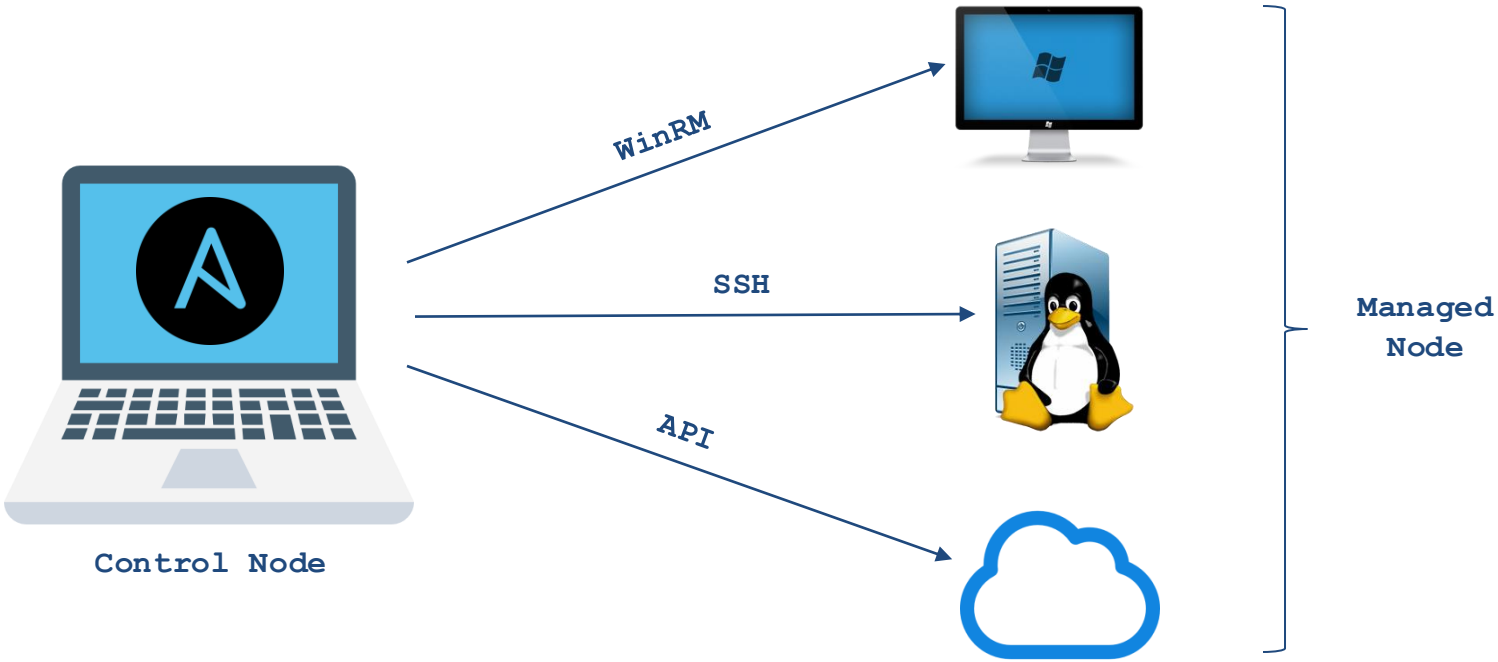
Ansible / exemple réel

```yaml
- name: Update nginx configuration restart the service
  hosts: all
  gather_facts: yes

  tasks:
    - name: Update nginx configuration file
      copy:
        src: nginx.conf
        dest: /etc/nginx/nginx.conf
      become: true

    - name: Restart nginx
      service:
        name: nginx
        state: restarted
      become: true

    - name: Display a message to indicate the playbook has completed
      debug:
        msg: "Nginx config updated and service restarted successfully!"
```

Collecter des info sur les hosts (dispo à travers les 'facts' 'ansible_facts.*')

Exécuter ce module en tant que 'root'

Ansible / topologie générale



Ansible / types de connexions

```yaml
- name: Demo play for Windows machines
  hosts: my_windows_machines
  gather_facts: yes
  connection: winrm

  tasks:
```

```yaml
- name: Demo play for Linux machines
  hosts: my_linux_machines
  gather_facts: yes
  connection: ssh

  tasks:
```

```yaml
- name: Demo play for cloud services
  hosts: my_cloud_environments
  gather_facts: yes
  connection: local

  tasks:
```

Ansible / exemple réel pour Azure

```yaml
# Playbook to create an Azure resource group
- name: Create a resource group in Azure
  hosts: localhost
  gather_facts: no
  connection: local

  tasks:
    - name: Create resource group
      azure.azcollection.azure_rm_resourcegroup:
        name: myResourceGroup
        location: francecentral
      register: rg_result

    - name: Display the result of resource group creation
      debug:
        var: rg_result
```

Ansible / exemple réel de playbook pour Azure

```yaml
# Playbook to create an Azure resource group
- name: Create a resource group in Azure
  hosts: localhost
  gather_facts: no
  connection: local

  environment:
    AZURE_TENANT: "{{ azure_tenant }}"
    AZURE_SUBSCRIPTION_ID: "{{ azure_subscription_id }}"
    AZURE_CLIENT_ID: "{{ azure_client_id }}"
    AZURE_SECRET: "{{ azure_secret }}"

  tasks:
    - name: Create resource group
      azure.azcollection.azure_rm_resourcegroup:
        name: myResourceGroup
        location: francecentral
      register: rg_result

    - name: Display the result of resource group creation
      debug:
        var: rg_result
```

Définition de variables d'environnement.
Ces 4 variables sont utilisées par les modules Azure

Ansible / exécution du playbook d'exemple pour Azure

```
ansible-playbook ./ansible/exemple_azure.playbook.yaml \
    --extra-vars "azure_tenant=MY_TENANT_ID" \
    --extra-vars "azure_subscription_id=MY_SUBSCRIPTION_ID" \
    --extra-vars "azure_client_id=MY_CLIENT_ID" \
    --extra-vars "azure_secret=MY_SECRET"

[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'

PLAY [Create a resource group in Azure] ****************************************************************************

TASK [Create resource group] ***************************************************************************************
changed: [localhost]

TASK [Display the result of resource group creation] ***************************************************************
ok: [localhost] => {
    "rg_result": {
        "changed": true,
        "contains_resources": false,
        "failed": false,
        "state": {
            "id": "/subscriptions/e32c78c4-2341-4ca4-a381-b2fa063a8e01/resourceGroups/myResourceGroup",
            "location": "francecentral",
            "name": "myResourceGroup",
            "provisioning_state": "Succeeded",
            "tags": null
        }
    }
}

PLAY RECAP *********************************************************************************************************
localhost                  : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Ansible / "client_id", "client_secret" ??

- **Service Principal** = utilisateur pour les applications
- Création depuis le portail Azure **"App registrations"**
- Création d'un **secret**

--> **client-id = identité** du Service Principal
--> **client_secret = mot de passe** du Service Principal

- Comme pour tout utilisateur/groupe, un Service Principal doit obtenir des permissions --> menu "Access Control (IAM)" au niveau d'un 'resource-group' ou d'une 'subscription'

Plan / Automatisation

> Infra-as-Code

> Outillage

- az CLI
- ansible
- **terraform**

> Collaborer sur azure

Automatisation / terraform

- **Idempotent**
- Réutilisation et partage de fonctionnalités ('**providers**')
- **Gestion** automatique des **dépendances**
- Planification vs. exécution
- Capacité à **détecter** les volontés de **suppression** et **les écarts** entre l'état 'voulu' et l'état réel du système --> **drift**
  (notion de 'terraform state')
- Structuration du code (**modules**)
- **templating** (propriétaire)

Automatisation / terraform – concepts principaux

Providers

- Chaque capacité de terraform est mise à disposition à travers un **'provider'** ex. azurerm, was, google, kubernetes, random, tls, acme, local, ...

```
terraform {
  required_version = ">= 1.0"

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"  # Registry source
      version = "~> 3.0"             # Version constraint
    }
    random = {
      source  = "hashicorp/random"
      version = "~> 3.0"
    }
  }
}
```

Automatisation / terraform – concepts principaux

Resources

- Représente un élément d'infrastructure ex. Azure virtual machine, AWS S3 bucket, certificat TLS

```
resource "RESOURCE_TYPE" "LOCAL_NAME" {
  # Configuration arguments
  argument1 = value1
  argument2 = value2

  # Nested blocks
  block_name {
    nested_argument = value
  }
}
```

Automatisation / terraform – concepts principaux

Resources

```
resource "azurerm_storage_account" "example" {
  # Required arguments
  name                     = "mystorageaccount"
  resource_group_name      = azurerm_resource_group.main.name
  location                 = "francecentral"
  account_tier             = "Standard"
  account_replication_type = "LRS"

  # Optional arguments
  enable_https_traffic_only = true
  min_tls_version           = "TLS1_2"

  # Nested block
  blob_properties {
    versioning_enabled = true
  }

  # Meta-arguments (special Terraform arguments)
  lifecycle {
    prevent_destroy = true
  }

  tags = {
    Environment = "Production"
    ManagedBy   = "Terraform"
  }
}
```

Automatisation / terraform – concepts principaux

Resources

```
resource "azurerm_resource_group" "main" {
  name     = "my-rg"
  location = "francecentral"
}

resource "azurerm_storage_account" "main" {
  name                = "mystorageacct"
  resource_group_name = azurerm_resource_group.main.name  # Reference!
  location            = azurerm_resource_group.main.location  # Reference!
  # ... other arguments
}
```

Automatisation / terraform – concepts principaux

Resources

```
resource "azurerm_storage_account" "example" {
  # ... normal arguments ...

  # Meta-argument: Create multiple instances
  count = 3
  name  = "storage${count.index}"

  # Meta-argument: Conditional creation
  count = var.environment == "prod" ? 1 : 0

  # Meta-argument: Create from map
  for_each = toset(["dev", "staging", "prod"])
  name     = "storage-${each.key}"

  # Meta-argument: Dependencies
  depends_on = [
    azurerm_resource_group.main
  ]

  # Meta-argument: Lifecycle rules
  lifecycle {
    create_before_destroy = true
    prevent_destroy       = true
    ignore_changes        = [tags]
  }
}
```

Automatisation / terraform – concepts principaux

Terraform 'state'

- "Base de données" de terraform. Il contiend (entre autres) l'association entre les ressource (fichiers .tf) et les éléments réels de l'infra
- Peut contenir des informations sensible (ex. mots de passe)
  --> ne doit pas être commité dans un dépôt git (.gitignore)
- Peut être stocké 'en local' ou sur un espace partagé (bucket S3, azure storage-account)
  --> chiffrement
  --> collaboration en équipe (gestion des opérations concurrentes)

Automatisation / terraform – concepts principaux

Terraform 'state'


```
{
  "version": 4,
  "terraform_version": "1.5.0",
  "serial": 3,
  "lineage": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "outputs": {
    "storage_account_name": {
      "value": "mystorageacct123",
      "type": "string"
    }
  },
  "resources": [
    {
      "mode": "managed",
      "type": "azurerm_storage_account",
      "name": "main",
      "provider": "provider[\"registry.terraform.io/hashicorp/azurerm\"]",
      "instances": [
        {
          "schema_version": 3,
          "attributes": {
            "id": "/subscriptions/.../resourceGroups/my-rg/providers/Microsoft.Storage/storageAccounts/mystorageacct123",
            "name": "mystorageacct123",
            "location": "francecentral",
            "account_tier": "Standard",
            "primary_blob_endpoint": "https://mystorageacct123.blob.core.windows.net/",
            "primary_access_key": "SENSITIVE_VALUE_REDACTED"
          }
        }
      ]
    }
  ]
}
```

Automatisation / terraform – concepts principaux

Gestion des dépendances

- Terraform établit un graphe de dépendances automatiquement pour déterliner l'ordre dans lequel exécuter les opérations
  --> simplicité
  --> parallélisation
  --> fonctionne … dans la plus part des cas

- terraform –graph | dot –Tsvg > graph.svg

- Pour les autres cas, on peut explicitement décrire des dépendances entre ressources

Automatisation / terraform – concepts principaux

Gestion des dépendances

- Pour les autres cas, on peut explicitement décrire des dépendances entre ressources

```
resource "azurerm_storage_account" "main" {
  name = "mystorageacct"
  # ...
}

resource "azurerm_role_assignment" "storage_access" {
  # This must happen AFTER storage account is fully ready
  scope                = azurerm_storage_account.main.id
  role_definition_name = "Storage Blob Data Contributor"
  principal_id         = var.app_principal_id

  # Explicit dependency to ensure storage is 100% ready
  depends_on = [
    azurerm_storage_account.main
  ]
}
```

Automatisation / terraform – concepts principaux

Workflow typique

```
Write Code
    ↓
terraform init      ← Download providers, initialize backend
    ↓
terraform plan      ← Preview changes, no modifications
    ↓
terraform apply     ← Execute changes, update state
    ↓
terraform destroy   ← Remove all managed infrastructure
```

Automatisation / terraform – concepts principaux

terraform init

```
$ terraform init

Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 3.0"...
- Installing hashicorp/azurerm v3.75.0...
- Installed hashicorp/azurerm v3.75.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

Automatisation / terraform – concepts principaux

terraform plan

```
$ terraform plan

Terraform will perform the following actions:

  # azurerm_resource_group.main will be created
  + resource "azurerm_resource_group" "main" {
      + id       = (known after apply)
      + location = "francecentral"
      + name     = "my-rg"
    }

  # azurerm_storage_account.main will be created
  + resource "azurerm_storage_account" "main" {
      + account_replication_type = "LRS"
      + account_tier             = "Standard"
      + id                       = (known after apply)
      + location                 = "francecentral"
      + name                     = "mystorageacct123"
      + primary_access_key       = (sensitive value)
      + primary_blob_endpoint    = (known after apply)
      + resource_group_name      = "my-rg"
    }

Plan: 2 to add, 0 to change, 0 to destroy.
```

Automatisation / terraform – concepts principaux

terraform apply

```
$ terraform apply

Terraform will perform the following actions:
  # ... plan output ...

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

azurerm_resource_group.main: Creating...
azurerm_resource_group.main: Creation complete after 2s [id=/subscri
azurerm_storage_account.main: Creating...
azurerm_storage_account.main: Still creating... [10s elapsed]
azurerm_storage_account.main: Creation complete after 24s [id=...]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

storage_account_name = "mystorageacct123"
```

Automatisation / terraform – concepts principaux

terraform destroy

```
$ terraform destroy

Terraform will perform the following actions:

  # azurerm_storage_account.main will be destroyed
  - resource "azurerm_storage_account" "main" {
      - name = "mystorageacct123" -> null
      # ...
    }

  # azurerm_resource_group.main will be destroyed
  - resource "azurerm_resource_group" "main" {
      - name = "my-rg" -> null
      # ...
    }

Plan: 0 to add, 0 to change, 2 to destroy.

Do you really want to destroy all resources?
  Enter a value: yes

azurerm_storage_account.main: Destroying... [id=...]
azurerm_storage_account.main: Destruction complete after 15s
azurerm_resource_group.main: Destroying... [id=...]
azurerm_resource_group.main: Destruction complete after 45s

Destroy complete! Resources: 2 destroyed.
```

Automatisation / terraform – concepts principaux

Autres commandes utiles

```
# Format code to canonical style
terraform fmt

# Validate configuration syntax
terraform validate

# Show current state
terraform show

# List resources in state
terraform state list

# Show specific resource
terraform state show azurerm_storage_account.main

# Show outputs
terraform output

# Show output value
terraform output storage_account_name

# Refresh state from real infrastructure
terraform apply -refresh-only

# Import existing resource
terraform import azurerm_resource_group.main /subscriptions/.../resourceGroups/existing-rg

# Replace resource
terraform apply -replace=azurerm_storage_account.main
```

Automatisation / terraform – exemple concrêt

```
# main.tf
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "main" {
  name     = "storage-rg"
  location = "francecentral"
}

resource "azurerm_storage_account" "main" {
  name                     = "mystorageacct12345"
  resource_group_name      = azurerm_resource_group.main.name
  location                 = azurerm_resource_group.main.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
}

output "storage_account_name" {
  value = azurerm_storage_account.main.name
}
```

Automatisation / terraform – exemple concrêt

```
$ terraform init
# download azurerm provier

$ terraform plan
# Shows: Plan: 2 to add, 0 to chage, 0 to destroy

$ terraform apply
# Creates resource-group, then storage-account
# Output: storage_account_name: "mystorageacct12345"
```

Automatisation / terraform – exemple concrêt

Apporter un changement

```
# Change LRS to GRS
resource "azurerm_storage_account" "main" {
  # ...
  account_replication_type = "GRS"  # Changed!
}
```

```
$ terraform plan
# Shows: ~ update in-place for storage account

$ terraform apply
# Updates storage account replication

$ terraform destroy
# Removes storage account, then resource-group
```

Plan / Automatisation

> Infra-as-Code

> Outillage

> Collaborer sur azure

Collaborer sur Azure

1. Créer un compte si il n'est pas trouvé dans 'Microsoft Entra Id'
2. Au niveau 'subscription' ou 'resource-group' affecter le droit 'Contributor' au nouvel utilisateur
   menu "Access control (IAM)"
   > Role assignments
   > Add role assignment
   > Privileged administrator roles
   > "Contributor"
   > "Next"
   > "User, group or service principal"
   > "+ Select members"
   > choisir l'utilisateur
   > "Select"
   > "Next"
   > "Review + assign"

Azure: travaux pratiques

Azure: TP – Ansible

- *Créer un Service Account et un client_secret*
- *Affecter le rôle 'Contributor' au Service Account au niveau de votre 'subscription'*
- *Installer Ansible*
- *Installer la "collection" Ansible pour Azure*
- *Créer un playbook Ansible qui crée un 'resource group'*
- *Mettre à jour le playbook pour y ajouter la création d'un 'Virtual Network'*

https://github.com/mdimonte/hexagonecloud/blob/main/azure/03-automation/README.md

Azure: TP – Terraform

- *Exercice 1: créer un fichier en local*
- *Exercice 2: créer un resource-group et un storage-account*
- *Exercice 3: créer un virtual-network et des subnets*
- *Exercice 4: déployer une application complète (web-app + database)*

https://github.com/mdimonte/hexagonecloud/blob/main/azure/03-automation/terraform/README.md#lets-start-with-exercices

Merci :)